

Lab 2: Estructures de dades lineals

1. Piles (stacks) i cues (queues) de la STL

El segon laboratori ens posa a prova amb *Data Structures* lineals. Utilitzarem l'equivalent de la Standard Template Library (STL) per als contenidors **Stack** i **Queue**. Si alguna vegada has vist apilar plats (l'últim plat que fiques és el primer a rentar) o posar-te en una fila al supermercat (el primer a arribar és el primer en sortir), acabes d'entendre al 100% que és una pila i una cua.

Interfícies: Stack i Queue

```
cpp [Interface Pila (Stack)] s.push(x) // Posa dalt. s.top() // Mira el dalt. s.pop()
// Treu de dalt. s.empty() // Està buida? s.size() // Elements.

cpp [Interface Cua (Queue)] q.push(x) // Posa a darrere. q.front() // Mira el davant.
q.pop() // Treu de davant. q.empty() // Està buida? q.size() // Elements.
```

2. Testos automàtics amb Doctest

El jutge realitza múltiples proves introduint dades al teu codi per esborrar si té algun error. Aquestes proves automàtiques s'emparen en C++ sota el framework **Doctest**. Un cop has programat la teva solució (ex. `reverse.cc`), se t'inclou al laboratori el **Doctest** i el **Makefile**. Només caldrà executar la comanda:

```
make test
```

3. Piles (Stacks)

Garanteixen el sistema **LIFO: Last In, First Out** (Últim a entrar, primer a sortir).

Exercici 1: Reverse

Consisteix en llegir números constants i imprimir-los a l'inrevés. L'apilament ho fa automàticament! Introduïrem els nombres un rere l'altre asimètricament al top de la Pila fins que no hi hagi dades a llegir de la cadena d'entrada (`while (cin >> n)`). Un cop plens, només anem imprimint els cim de l'actual recurs(`top()`) i desapilem (`pop()`) fins fons.

Codi solució: reverse.cc

```
“cpp [p1-reverse/reverse.cc] #include using namespace std; #include “stack.hh” using namespace pro2;
void reverse(istream& in, ostream& out) { Stack s; int n; while (in » n) { s.push(n); }

// Anem desapilant i cridant el TOP per extreure en invers
while (!s.empty()) {
    out << s.top();
    s.pop();
    if (!s.empty()) out << " ";
```

```

}
out << endl;
}

```

[Simulació interactiva disponible a la web]

Exercici 2: Validar Parèntesis

Apilem només les obertures (`(` o `[`). Quan arriba un tancament (`)` o `]`), comprovem si quadra amb el

****Codi solució: parenthesis.cc****

```

```cpp [p2-parenthesis/parenthesis.cc]
#include <iostream>
using namespace std;
#include "stack.hh"
using namespace pro2;

void parenthesis(istream& in, ostream& out) {
 Stack<char> s;
 char c;
 int pos = 1;

 // Llegim ignorant espais (in >> c) fins topar-nos pel caràcter punt delimitador pur
 while (in >> c and c != '.') {
 // Enfilem obertures al front pur
 if (c == '(' or c == '[') {
 s.push(c);
 }
 else if (c == ')' or c == ']') {
 // Tancament excessiu o previ no quadra
 if (s.empty()) {
 out << "Incorrecte " << pos << endl;
 return;
 }
 char top = s.top();
 // Avaluem matching i el desfem segurament:
 if ((c == ')' and top == '(') or (c == ']' and top == '[')) {
 s.pop();
 } else {
 out << "Incorrecte " << pos << endl;
 return;
 }
 }
 pos++;
 }
}

```

```

 // Finalitzat, si ha restat alguna obertura lliure pendants considerarem error
 if (s.empty()) out << "Correcte\n";
 else out << "Incorrecte " << pos << endl;
}

```

*[Simulació interactiva disponible a la web]*

### Exercici 3: Recursivitat simulada amb Piles

L'ordinador utilitza una pila oculta (el Call-Stack) per processar funcions recursives. Aquest exercici ens demostra com qualsevol funció recursiva del tipus  $f(n-1)$  pot traduir-se a codi iteratiu. Al bucle iteratiu prenem el `.top()`, i si compleix la condició de viabilitat ( $v > 0$ ), simulem la creació teòrica d'activitats afegint manualment dues operacions més petites a la pila.

#### Codi solució: recursivitat.cc

```

“cpp [p3-recursivitat/recursivitat.cc] #include using namespace std; #include “stack.hh” using namespace
pro2;

void escriu(int n, ostream& out) { Stack s; s.push(n);

// Iterant contínuament fins haver desapilat per pur tota acció virtual
while (!s.empty()) {
 int v = s.top();
 s.pop();

 if (v > 0) {
 out << ' ' << v;
 // Instanciem instint base C++, cap endarrere ja que el següent pas voldrà
 // fer 'pop' i consumir el "MÉS NOU"
 s.push(v - 1);
 s.push(v - 1);
 }
}
}
}

```

*\*[Simulació interactiva disponible a la web]\**

---

### ## 4. Cues (Queues)

Les cues garanteixen l'estàndard **\*\*FIFO: First In, First Out\*\***. Usarem ``front`` al revés per albirar excl

### ### Exercici 5: La Patata Calenta

Resol el problema cíclic de Josephus de  $N$  participants utilitzant només una Cua. Per simular passades

**\*\*Codi solució: patata.cc\*\***

```

```cpp [p5-patata-calenta/patata.cc]
void patata_calenta(istream& in, ostream& out) {
    int N, k;
    if (in >> N >> k) {
        Queue<int> q;
        for (int i = 1; i <= N; ++i) {
            q.push(i); // Noms i gent dins del joc
        }

        bool first = true;
        while (q.size() > 1) { // Fins que sobrevisqui només pur 1 individu
            // Fem K girs o "passos de patates calents" cap a fi del cicle
            for (int i = 0; i < k; ++i) {
                int front = q.front();
                q.pop();
                q.push(front);
            }

            if (!first) out << " ";
            // La pobra anima davantera que acaba tocant rep l'expulsió immediata
            out << q.front();
            q.pop();
            first = false;
        }

        if (!first) out << endl;
        if (q.size() == 1) {
            out << "Supervivent: " << q.front() << endl;
        }
    }
}
}

```

[Simulació interactiva disponible a la web]

Exercici 6: Comptador Recents (Sliding Window)

Atès un llinar base de tolerància de temps T , quants anteriors segueixen “vius” passat el temps? Aquest patró es coneix com a finestra lliscant (“Sliding Window”) i és la principal propietat vitalícies útil d’una cua. Al llegir un nou instant (*actual*), caduquem els més vells utilitzats mirant el `front()`: Si és menor que $actual - T$, ho podem donar per prescrit. Acabada la purga seqüencial, demanem simple quin `size()` actiu tenim!

Codi solució: `recents.cc`

```

“`cpp [p6-compta-recents/recents.cc] void compta_recents(istream& in, ostream& out) { int N, T; if (in » N
» T) { Queue q; bool first = true;

    for (int i = 0; i < N; ++i) {
        int t;
        in >> t;
        q.push(t);

        // Evaluador extern caducant antics vells emmagatzemats d'espant
        // que queden enlloc pel rang de temps i de Cua fora:
        while (!q.empty() && q.front() < t - T) {
            q.pop();

```

```

    }

    if (!first) out << " ";
    out << q.size(); // Mostra base actius vius
    first = false;
}
out << endl;
}
} ““

```

[Simulació interactiva disponible a la web]